

Protocole expérimental et méthodologie d'évaluation

1 Objectif de l'étude

L'objectif de cette expérimentation est de comparer plusieurs méthodes d'affichage d'un volume important d'informations dans Unity, en portant une attention particulière à l'impact des mécanismes réseau sur les performances de rendu, de calcul et de synchronisation.

2 Protocole expérimental

Afin de garantir la reproductibilité des résultats, un protocole unique est appliqué à l'ensemble des solutions étudiées.

2.1 Phase 1 – Préparation du test

Cette phase correspond à l'initialisation de l'environnement de test ainsi qu'à la mise en place des conditions nécessaires au lancement de l'expérimentation.

2.2 Phase 2 – Instanciation des entités

Les entités sont instanciées progressivement par vagues successives selon des paramètres identiques pour tous les scénarios. Cette phase permet d'évaluer la capacité du système à gérer l'apparition simultanée d'un grand nombre d'objets.

2.3 Phase 3 – Mouvement des entités

Les entités précédemment instanciées sont progressivement mises en mouvement jusqu'à atteindre 100 % de la population simulée. Cette phase vise à mesurer l'impact de la simulation dynamique sur les performances.

Note. Dans la version actuelle de l'expérimentation, les phases d'instanciation et de mouvement sont fusionnées dans un scénario unique (*Spawn + Move*). La distinction est conservée uniquement pour assurer la compatibilité avec les expérimentations antérieures.

3 Périmètre de l'analyse réseau

L'étude ne porte pas sur le nombre maximal de clients simultanément connectés. Les travaux antérieurs indiquent que l'évolution des performances suit une tendance relativement prévisible avec l'augmentation du nombre de clients.

L'analyse se concentre principalement sur :

- la capacité d'affichage des données pour un client unique ;
- la charge induite côté serveur ;
- la charge induite côté client ;
- les coûts de synchronisation réseau.

4 Paramètres expérimentaux

Paramètre	Valeur
Nombre total d'entités	20 000
Taille d'une vague d'instanciation	400
Délai entre deux vagues	10 s
Entités mises en mouvement par palier	10 %
Délai avant activation du mouvement	5 s
Durée de préparation initiale	10 s
Temps d'attente entre phases	10 s
Temps avant fermeture automatique	10 s
Mode d'exécution	Spawn + Move

4.1 Définition d'une entité

Une entité correspond à un cube Unity standard composé de 12 triangles, utilisant une texture de base et sans éclairage dynamique.

Dans les versions réseau, chaque entité possède un `NetworkObject` ainsi qu'un composant de synchronisation de position dépendant de l'implémentation étudiée.

Les entités se déplacent à vitesse constante de 5 m/s sur chaque axe et effectuent simultanément une rotation uniforme.

5 Groupes d'étude

5.1 Configuration locale

Cette configuration ne comporte aucune couche réseau apparente et permet d'évaluer les limites intrinsèques du moteur de rendu.

5.2 Configuration réseau

Cette configuration étudie le comportement des performances côté serveur et côté client dans un contexte client-serveur réel.

6 Mesures collectées

6.1 Performances de rendu

- Images par seconde (FPS) ;
- Temps CPU par image (ms) ;
- Temps GPU par image (ms).

6.2 Consommation mémoire

- Mémoire utilisée (MB).

6.3 Performances réseau

- RTT (Round Trip Time) ;
- RTT calculé à partir des RPC.

6.4 Charge réseau

- Débit montant ;
- Débit descendant ;
- Volume total envoyé ;
- Volume total reçu ;
- Nombre de paquets envoyés ;
- Nombre de paquets reçus.

6.5 Analyse du trafic

Une capture indépendante du trafic réseau est réalisée afin d’analyser :

- le nombre de paquets par seconde ;
- le volume de données échangé ;
- les volumes cumulés ;
- l’évolution de la charge réseau.

7 Méthode d’analyse

Les indicateurs sont étudiés en fonction du nombre d’entités présentes dans la scène afin d’identifier :

- les limites serveur et client ;
- l’impact de la couche réseau ;
- les goulots d’étranglement ;
- l’évolution de la consommation des ressources ;
- les limites matérielles des plateformes.

8 Typologie des clients

Deux catégories de clients sont étudiées :

1. Ordinateur personnel (PC) ;
2. Casque autonome Meta Quest 3.

Les résultats sont analysés séparément pour chaque plateforme.

9 Configuration matérielle

9.1 Serveur

Les expérimentations FishNet et Netcode for GameObjects utilisent un serveur dédié Dell Precision 7550 exécuté en mode headless.

Composant	Caractéristiques
Système	Windows 11 Enterprise 64 bits
CPU	Intel Core i7-10850H
RAM	32 Go
GPU	NVIDIA Quadro RTX 3000

API	DirectX 12
Résolution	1920 × 1080

9.2 Client PC

Composant	Caractéristiques
Système	Windows 11 Enterprise 64 bits
CPU	Intel Core Ultra 9 285K
RAM	128 Go
GPU	NVIDIA RTX 5000 Ada Generation
API	DirectX 12
Résolution	2560 × 1440

9.3 Client réalité virtuelle

Les tests immersifs sont réalisés avec un Meta Quest 3.

10 Infrastructure réseau

Pour FishNet et Netcode for GameObjects, le serveur et les clients sont connectés à un réseau local via un routeur Netgear R6020.

Pour Photon, les communications transitent par l’infrastructure cloud du fournisseur. Les mesures de latence obtenues ne sont donc pas directement comparables à celles obtenues avec les architectures locales.

11 Configuration réseau

Les expérimentations locales reposent sur :

- un serveur headless ;
- un client PC ou Meta Quest ;
- un routeur Netgear R6020 ;
- des conditions réseau stables et contrôlées.

Dans le cas de Photon, les communications transitent par Internet et utilisent l’infrastructure cloud du fournisseur.